AIORI HACKATHON 3 • CYBER SECURITY

A3-PS013-TC319

TraceKit: tamper-proof incident timelines

PS-013 Incident Timeline and Evidence Collection for Small Teams

Team Hash Hunters
Palak Deval (Team Leader)
devaluma920@gmail.com
Members: Shruti Yadav, Yogesh Asnora

October 14, 2026

Change one character, change the fingerprint

ORIGINAL • SHA-256 of “TraceKit”

```
551fe3b6c4877c008bcb433424a9eb72
d479e34ea753949fa956bd8c2c52aee1
```

MODIFIED • SHA-256 of “Tracekit”

```
3228c7df73472212ef5b5b5ed062fd13
da678182bc318a4cd86ccd4752e258f8
```

Every file is sealed on upload, so any edit is exposed.

Problem Description

What happened? **When?** **Who did it?** **Can we prove it?**

After an attack the questions are always the same, but the evidence is scattered:

Server logs
sit on the affected machine

Screenshots
sit on someone's phone

Emails & chats
in inboxes and WhatsApp groups

No clear sequence

Logs use different time formats, so nobody can easily say what happened first.

No proof of integrity

A file can be edited, by accident or on purpose, and nobody can show it is original.

No affordable tools

Forensic software is costly and needs trained experts that colleges, clinics and startups lack.

India's DPDP Act 2023 expects organisations to handle personal-data breaches responsibly [5]. Without reliable evidence, a small team cannot investigate, report or defend itself.

Background

Chain of custody

A record of who handled each piece of evidence, and when. Forensic practice treats it as essential [1][4].

Cryptographic hash (SHA-256)

A 256-bit fingerprint of a file. Any edit, even one character, produces a different value [3].

Timeline reconstruction

Merging events from many sources into one time-ordered sequence of what happened [2].

Real-world example

Police	TraceKit
Collect scene evidence	Upload logs, screenshots, notes
Arrange it in time order	Merge into one timeline
Seal it against tampering	SHA-256 seal + tamper check

Small teams need all three, without the cost or the training.

Literature & RFC Review

Standards we build on

RFC 3227 [1]	Evidence collection: record everything and keep custody documented
NIST SP 800-86 [2]	Forensics in incident response: collect, examine, analyze, report
FIPS 180-4 [3]	SHA-256 defines the fingerprint that seals every file
ISO/IEC 27037 [4]	Identify, collect, acquire and preserve digital evidence
RFC 5424, 3339 [6][7]	Syslog and timestamp formats our parser normalizes

Existing tools and the gap

Autopsy, FTK Imager

Disk-forensics focus, built for trained examiners

Plaso + Timesketch

Powerful timelines, but heavy setup for a small team

SIEMs (Splunk, ELK)

Built for continuous monitoring, heavy to run and tune

The gap: no lightweight tool that combines custody, a timeline and a tamper check for non-experts.

Proposed Solution

TraceKit: a simple web tool that seals evidence and turns it into a tamper-evident incident timeline.



1. Upload

Logs, screenshots, notes



2. Seal

SHA-256, uploader, time



3. Timeline

All formats, one order



4. Red Flags

Suspicious patterns found



5. Attack Story

2–3 lines, plain English



6. Tamper Check

Re-upload: Original or Modified



7. Report

One-click PDF

SAMPLE TIMELINE

2:00 AM	50 failed logins	RED FLAG
2:05 AM	Successful login	RED FLAG
2:10 AM	customers.xlsx deleted	RED FLAG

ATTACK STORY

"Between 2:00 and 2:04 AM, 50 failed logins were detected. At 2:05 AM the attacker logged in successfully and deleted customers.xlsx at 2:10 AM."

Optimization Proposed

Our goal: anyone can build a reliable incident record in minutes instead of days.

Aspect	Before: manual process	With TraceKit
Building the record	Days of manual stitching	**Minutes, in one workflow**
Evidence location	Phones, inboxes, servers	**One case, one place**
Time ordering	Compare mixed formats by hand	**Auto-merged chronological timeline**
Integrity proof	“Trust me”, no proof	**SHA-256 seal + tamper check**
Reading the incident	Thousands of log lines	**Red flags + plain-English story**
Reporting	Written by hand	**One-click PDF with hashes**
Cost and skills	Paid tools, trained experts	**Free tools, a normal laptop**

Solution Architecture & Design

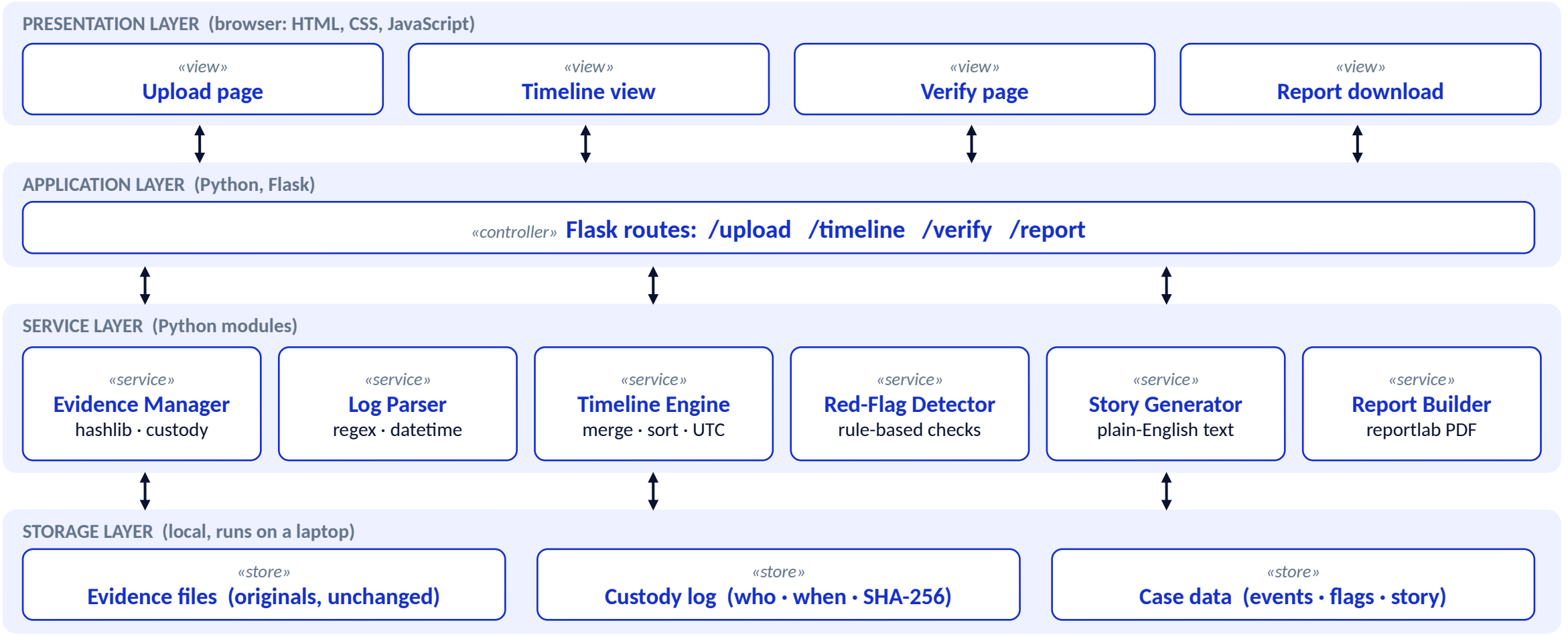


Figure: System architecture (UML component view)

Data Flow Diagram

Level 1 view: how evidence flows from upload to the final report (same 7-step flow as slide 5).

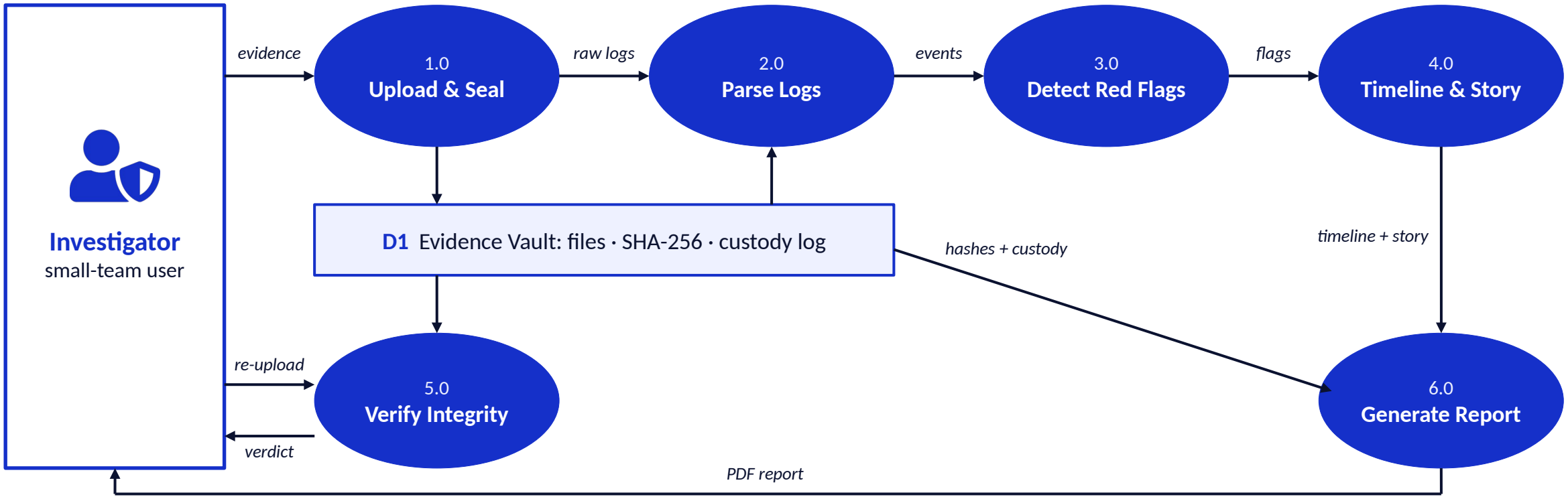


Figure: Data Flow Diagram (Level 1)

Deployment Timeline

Incremental delivery: a working demo first, then a pilot, then hardening.

1

Core build

Week 1

- Upload and SHA-256 seal
- Chain-of-custody log
- Parsers for 2–3 log formats

2

Detect & explain

Week 2

- Red-flag rules
- Attack Story summary
- Tamper check

3

Report & test

Week 3

- One-click PDF report
- Test on sample attack logs
- Demo-ready build

4

Pilot

Weeks 4–6

- Try with a college lab or clinic
- Collect feedback
- Add more log formats

5

Harden & scale

After pilot

- Login and user roles
- Docker one-command deploy
- Trusted timestamping

Methodology: agile weekly sprints, every milestone tested on realistic sample logs, and a deployment that runs on a normal laptop with Python and Flask. No paid services needed. Timeline is proposed.

Bibliography

[1] D. Brezinski and T. Killalea. “Guidelines for Evidence Collection and Archiving.” RFC 3227, IETF, 2002.

[2] K. Kent, S. Chevalier, T. Grance and H. Dang. “Guide to Integrating Forensic Techniques into Incident Response.” NIST SP 800-86, 2006.

[3] NIST. “Secure Hash Standard (SHS).” FIPS PUB 180-4, 2015.

[4] ISO/IEC 27037:2012. “Guidelines for identification, collection, acquisition and preservation of digital evidence.”

[5] Government of India. “The Digital Personal Data Protection Act, 2023.” Act No. 22 of 2023.

[6] R. Gerhards. “The Syslog Protocol.” RFC 5424, IETF, 2009.

[7] G. Klyne and C. Newman. “Date and Time on the Internet: Timestamps.” RFC 3339, IETF, 2002.

Thank You for Your Attention!

Every log tells a story. Hash Hunters make it tamper-proof.



TraceKit • Team Hash Hunters • PS-013